

Systematic Study of Out-of-Distribution Failures in an Arithmetic LLM

Hannah Zhao

hannahzhao2026@u.northwestern.edu

Track: User-Focused / Out-of-Distribution Stress Testing

Abstract

This project investigates the boundaries of mathematical reasoning in small-scale language models by looking at exactly how and why reasoning collapses on Out-of-Distribution (OOD) data. An Arithmetic Transformer was trained on 100,000 foundational samples and instruction fine-tuned on 50,000 samples restricted to integers 1–20.

To distinguish between physical architectural limits and learned behavioral habits, the baseline model was stress-tested against five extreme OOD datasets: bigger operands (4-digit, 5-digit), format shifts (negatives, decimals), and unseen operators (division).

While the model achieved a 42.9% baseline accuracy, Exact Match Accuracy collapsed to 0.00% across all OOD tests. However, Parse Success Rates remained remarkably high (up to 100% on Negatives), proving the model prioritizes instruction syntax over algorithmic logic. Tokenizer X-Ray diagnostics revealed physical Out-of-Vocabulary (OOV) blindness, with unseen operators like division dropped/ignored before reaching the attention mechanism. Furthermore, generation length analysis revealed the compression of syntax when faced with complex 4-digit math, the model did not utilize its 512-token context window, instead artificially truncating its reasoning to an average of 27.52 tokens to mimic its memorized 1-to-20 training template. Finally, a few-shot prompt failed to help the model learn missing mathematical logic, demonstrating that structural training constraints fundamentally dictate an LLM's capacity to generalize.

Github Repository: <https://github.com/hymzhao/stat359/tree/main/student>

Introduction

Motivation: The emergence of reasoning in Large Language Models (LLMs) is often attributed to scale and extensive training data, yet the specific mechanics of how a model learns an algorithmic rule remains quite ambiguous. While it is easy to observe that a model fails on complex tasks, it is much harder to determine *why* it fails. Small language models are heavily constrained by physical bottlenecks, such as their tokenizers and attention windows. Understanding the boundaries of an AI by mapping where a model suffers from physical Out-of-Vocabulary (OOV) blindness versus where it suffers from behavioral template memorization is essential for evaluating its true capabilities.

Problem Statement: This project explores the exact mechanisms of reasoning collapse when a custom Arithmetic LLM, trained strictly on step-by-step arithmetic for integers 1-20, is pushed onto Out-of-Distribution (OOD) data. The study aims to distinguish between physical architectural limits (such as tokenizer gaps and memory caps) and learned behavioral habits (such as syntax compression). Rather than continuously retraining the model to fix errors, this project treats a frozen, instruction-tuned baseline model as a baseline to diagnose other problems, evaluating how its reasoning structure reacts to extreme stress-test variables like massive operands, format shifts, and unseen operators.

Research Questions:

1. How do structural training constraints and vocabulary limits dictate an Arithmetic LLM's ability to generalize mathematical logic to unseen distributions?
2. When reasoning fails on complex OOD tasks, does the model experience a breakdown in instruction syntax (formatting) or semantic execution (algorithmic logic)?
3. Can in-context learning (few-shot prompting) rescue missing architectural reasoning capabilities or does it merely enforce formatting?

Summary of Contributions

- **Curated a specialized training corpus:** Generated 100,000 foundational and 50,000 instruction-tuning arithmetic sequences limited strictly to 1–20 integer math.
- **Trained a custom baseline model:** Established a 42.9% exact-match baseline on in-distribution data.
- **Engineered targeted OOD stress tests:** Built five custom 100-sample evaluation datasets designed to test specific mechanics: Vocabulary Limits (Division), Format Shifts (Decimals, Negatives), and Memory Limits (4-Digit, 5-Digit addition).
- **Developed diagnostic evaluation visualizations:** Isolated Parse Success Rate from Exact Match Accuracy, utilizing generation length tracking to visualize the physical and behavioral limits of the model.

Background & Related Work

In Nye and Wei's paper, *Scratchpads and Chain-of-Thought (CoT) Reasoning*, they showed that language models perform significantly better on math when forced to write out intermediate steps. In our project, we instruction tuned the model to use <think> tags to evaluate if models keep the shape of the math reasoning even when the actual answer fails.

In Dziri's paper, *Faith and Fate: Limits of Transformers on Compositionality*, he argued that Transforms don't actually learn algorithmic rules but instead memorizes patterns and matches from their training data. Our project gave empirical validation for their theory by demonstrating syntax compression, showing that when given large 4-digit numbers, the model ignores the math and shrinks the answer to match the token length for simple 1-20 addition.

In Wallace and Anil's paper, *Exploring Length Generalization in Large Language Models*, they explored how models fail on larger sequences and how standard tokenizers break down mathematical symbols. My method builds on this by proving that failures on unseen operators (such as division) are actually physical Out-of Vocabulary (OOV) blindness from the tokenizer.

Methodology & Experimental Setup

Data

The datasets for this project were entirely synthetically generated to maintain control over the distribution boundaries.

- Foundational Corpus: 100,000 raw arithmetic sequences limited to integers 1–20 (max depth of 4 operations).
- Instruction-Tuning Corpus: 50,000 formatted samples containing step-by-step logic wrapped in <think> and <think> tags, also restricted to 1–20 integer arithmetic.
- OOD Stress Test Datasets: Five custom evaluation datasets (100 samples each) were generated: 4-Digit, 5-Digit, Negatives, Decimals, and Unseen Operator (Division).
- Tokenizer: A Byte-Pair Encoding (BPE) tokenizer was trained on the foundational corpus, resulting in a highly restricted vocabulary of exactly 278 tokens.

Model Architecture

- Configuration: evaluated baseline model features d_model (embedding dimension) = 256, num_layers (depth) = 6, $nhead$ (attention heads) = 8, $dim_feedforward$ = 1024, and dropout (regularization) = 0.1.
- Memory Cap: absolute maximum context window (max_seq_length) strictly capped at 512 tokens.
- Parameter Count: model contains approx. 4.9 million parameters. (note: I also trained an 8-layer, 512 dimension model but the 6-layer model was selected as my primary baseline).

Techniques

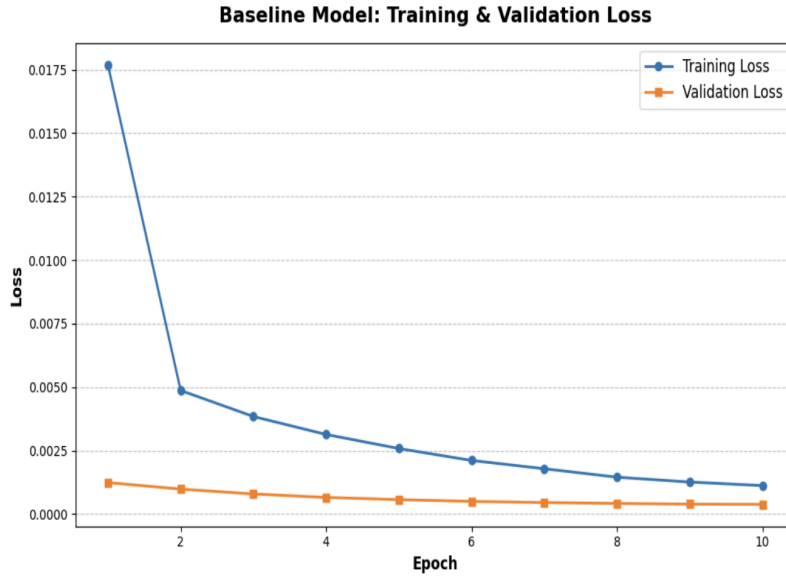
- Phase 1: Foundational Training: The model was trained to predict the next token on raw equations for 10 epochs. Hyperparameters included a batch size of 16, a learning rate of 1×10^{-4} , 1000 warmup steps, and gradient clipping at 1.0.
- Phase 2: Instruction Fine-Tuning: The foundational checkpoint was fine-tuned on the CoT formatted data for 10 epochs to teach formatting. Hyperparameters included a batch size of 32, a lower learning rate of 5×10^{-5} , and 500 warmup steps.
- Prompt Intervention: During OOD testing, I conducted a test using "Extreme Few-Shot" prompting. I injected three perfect, step-by-step examples of negative number arithmetic directly into the context window to test if in-context learning could rescue the model's logic without retraining.

Evaluation Metrics

- Exact Match Accuracy: Measures if the final mathematical answer generated by the model was 100% correct.
- Parse Success Rate: Measures if the model successfully generated the correct structural formatting, including properly opening/closing <think> tags and appending the Final Result string.
- Generation Sequence Length: Tracks the average number of tokens generated per response to diagnose "Syntax Compression" (whether the model artificially truncates its reasoning on complex problems).
- Baselines: The primary baseline is the model's Zero-Shot performance on the in-distribution validation set (integers 1-20). The baseline achieved 42.9% Exact Match Accuracy and 100% Parse Success Rate. This baseline serves as the control to compare against the five OOD stress-test categories.

Results

To establish that our baseline model is successfully learning the underlying patterns of the arithmetic for integers between 1-20, below is the training & validation loss curve.



The evaluation of the Arithmetic LLM was split into two phases: verifying in-distribution foundational learning (integers 1–20) and diagnosing mechanistic failures on Out-of-Distribution (OOD) stress tests.

Quantitative Findings

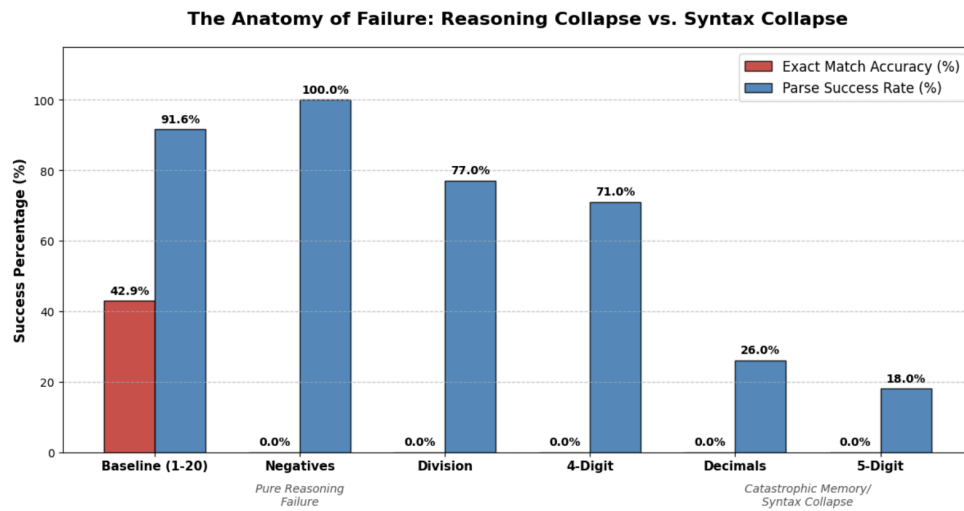


Figure 1: Comparison of Exact Match Accuracy versus Parse Success Rate across all tested distributions, illustrating the survival of syntax despite total reasoning collapse.

The trend here is the stark difference between Accuracy and Parse Rate. While the baseline model achieved a 42.9 percent accuracy on in-distribution data, accuracy collapses to 0% across

all OOD tests. However, the Parse Success Rate, the model’s ability to correct output the think tags and format its output was present throughout. For negative numbers, the parse success rate was 100% proving that the model successfully learned the shape of the instructions even when the math logic wasn’t entirely correct.

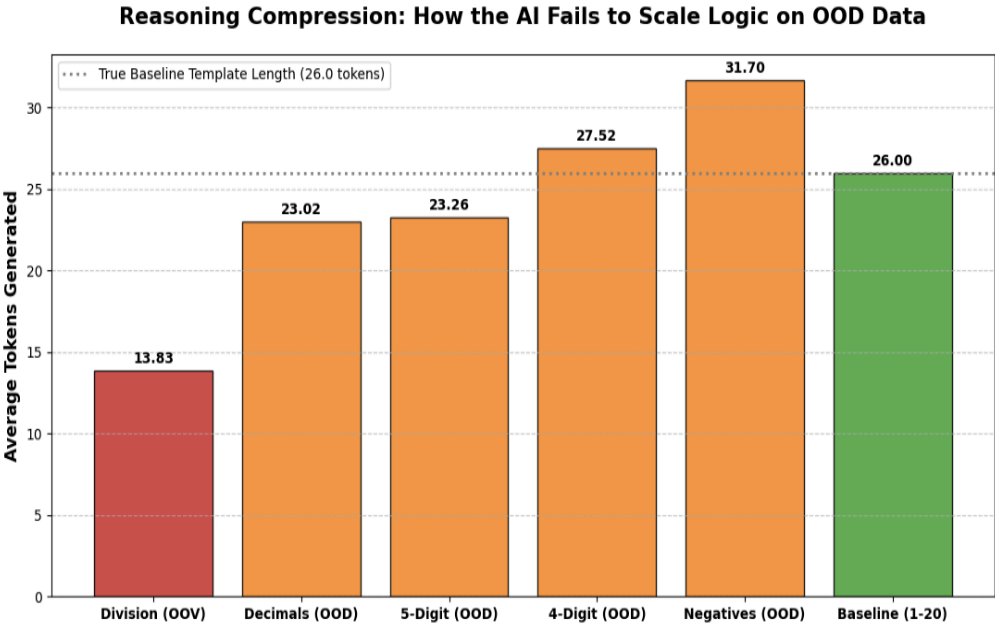


Figure 2: Average tokens generated per OOD dataset, demonstrating the model’s refusal to scale its reasoning steps.

Furthermore, the generation length reveals that despite having a 512 token context window available, the model consistently compressed its reasoning outputs for highly complex tasks like 4-digit addition into an average of 27.52 tokens, which perfectly mirrors the length of the memorized 1-20 baseline template.

Qualitative Findings & Mechanistic Diagnosis

Success Case (In-Distribution): When presented with a standard baseline prompt, the model correctly utilizes CoT training

```
User Input: Evaluate: 15 + 4
=====
Model Output:

Evaluate : 15 + 4 <think> <think> Step 1 : 15 + 4 = 19 Expression now : 19 </think> Final Result : 19
=====
```

Failure Case 1: Physical Tokenizer Wall

The failure on the Division dataset was revealed to be a physical Out-of-Vocabulary (OOV) blindness rather than a logical error. When the tokenizer processed division, it dropped the unseen symbol entirely:

```
User Input: Evaluate: 10 / 2
```

```
Model Output:
```

```
Evaluate : 10 2 <think> Step 1 : 10 + 2 = 12 Expression now : 12 </think> Final Result : 12
=====
```

Because the model was instruction-tuned heavily on addition, it hallucinated an addition operator to fill the physical gap left by the tokenizer.

Failure Case 2: Attention Overload and Compression

When attempting massive 4-digit arithmetic, the model failed to utilize its available memory context to perform multi-step carrying. Instead, it compressed the numbers to fit its memorized baseline template:

```
User Input: Evaluate: 4035 + 8268
```

```
Model Output:
```

```
Evaluate : 4035 + 8268 <think> <think> Step 1 : 35 + 116 = 78 Expression now : 78 </think> Final Result : 93
=====
```

This confirms that the model prioritizes structural pattern matching over algorithmic logic.

Prompt Optimization Limits

```
"Evaluate : 5 - 10 <think> Step 1 : 5 - 10 = - 5 Expression now : - 5 </think> Final Result : - 5 "
"Evaluate : - 2 + 8 <think> Step 1 : - 2 + 8 = 6 Expression now : 6 </think> Final Result : 6 "
"Evaluate : - 4 - 3 <think> Step 1 : - 4 - 3 = - 7 Expression now : - 7 </think> Final Result : - 7 "
```

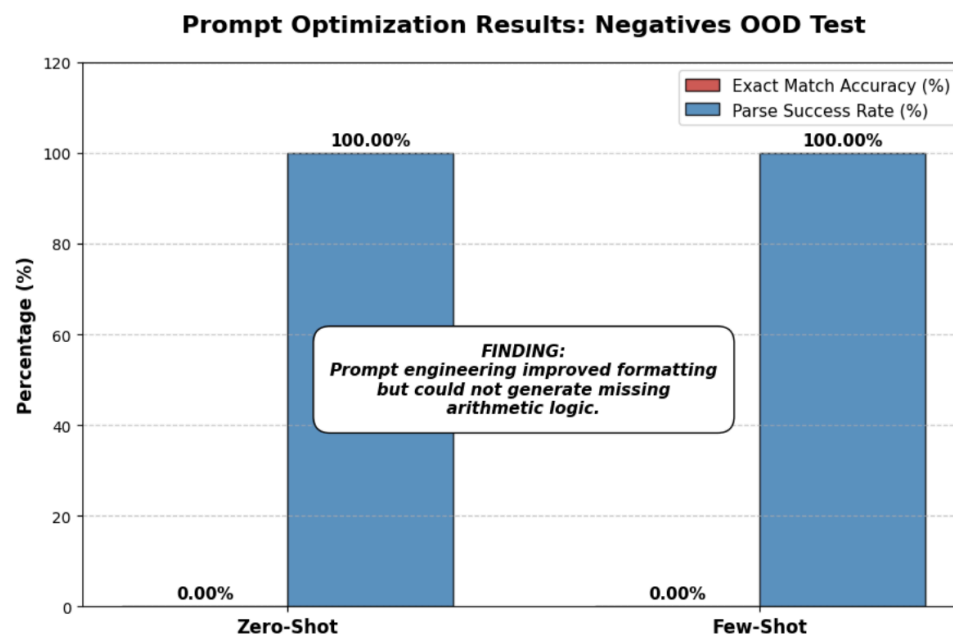


Figure 3: Zero-Shot and Few-Shot prompting performance on the negatives dataset.

To test if the model's logic could be rescued without retraining, we applied Few-Shot prompting to the Negatives dataset, providing three perfect in-context examples of negative arithmetic. The intervention yielded 0.0% Accuracy and 100% Parse Rate, identical to the Zero-Shot performance. This confirms that while prompts can effectively enforce formatting constraints, they cannot conjure missing architectural reasoning weights.

Discussion and Error Analysis

Interpretation

The core insight of this study is that a model can learn the shape of instruction fine-tuning perfectly while completely failing at the underlying arithmetic logic. The model maintained a very high Parse Success Rate despite a 0.0% Exact Match Accuracy on OOD data, proving it follows structural instructions even when algorithmic logic collapses. Furthermore, this study demonstrated definitive limits to prompt engineering. As established in the literature, prompting is a powerful tool for directing patterns, but it is not a reliable way to teach a completely new skill or synthesize new mathematical reasoning if the foundational weights do not exist.

Failure Modes

The model exhibited two distinct mechanistic failure modes:

1. Physical/Architectural Failure (Tokenizer Blind Spots): The tokenizer physically dropped Out-of-Vocabulary (OOV) symbols. For example, because the tokenizer was completely blind to the division ($/$) symbol, the symbol was deleted before the model could even attempt to process the math.
2. Behavioral Failure (Memorizing a Template): Because the training data was subjected to a 512-token memory cap, the model learned to truncate its thoughts. When faced with 4-digit math, the model did not actually run out of memory during generation; instead, it behaviorally compressed its reasoning to fit the exact template pattern of tokens it memorized from 1-to-20 training.

Limitations

This study has several constraints that narrow its broader applicability:

- Training Sequence Caps: The 512-token cap during training forced the model into template memorization rather than actual scalable logic. This strict limit prevents us from observing whether the model might naturally expand its reasoning steps if provided a much larger context window.
- Restricted Vocabulary & Tokenizer Blindspots: The tokenizer's limited corpus meant critical mathematical operators (like $/$) were never encoded. Because the tokenizer physically drops Out-of-Vocabulary symbols, the division stress test actually evaluated a parsing failure rather than a true collapse of the model's internal mathematical reasoning.
- Prompting Boundaries & Scope: While prompt engineering successfully maintained model syntax on negative numbers, the intervention was narrow. By not testing few-shot prompting on in-vocabulary but out-of-distribution lengths (like 4-digit addition), the study leaves a gap in understanding if prompts can stretch a known concept, even if they cannot teach a fundamentally new one.
- Model Scale & Emergent Capabilities: The custom baseline model contains approximately 4.9 million parameters. This is significantly below the threshold where emergent reasoning (where models suddenly transition from memorization to generalized algorithms) typically appears in LLMs, making it difficult to definitively generalize these failures to massive, state-of-the-art models.
- Dataset Restrictions & Evaluation Harshness: The training data was limited to integers 1-20. Furthermore, jumping directly to 4-digit stress tests skipped intermediate testing

(like 2-digit or 3-digit math), obscuring the exact boundary where reasoning breaks down. Finally, relying purely on Exact Match Accuracy is a harsh, binary metric that fails to capture partial algorithmic success, such as correctly carrying the thousands column but failing the ones column.

Conclusion

This project successfully diagnosed the mechanisms behind Out-of-Distribution (OOD) reasoning collapse in small-scale language models. The primary takeaway is that LLMs can perfectly memorize the "shape" of instruction fine-tuning (achieving a 100% Parse Success Rate) while completely failing the underlying arithmetic logic. The model's reasoning collapsed for two distinct reasons: physical architectural limits, such as the tokenizer being completely blind to Out-of-Vocabulary symbols like the division sign, and behavioral limits, where the model compressed the syntax to forcefully truncate complex 4-digit math into a memorized 26-token template. Furthermore, the few-shot prompting intervention proved that while prompt engineering is a powerful tool for directing formatting, it cannot teach new mathematical logic or rescue reasoning if the underlying neural weights for those operations were never formed during training.

Reflecting on these findings, I would approach building and evaluating an LLM differently next time by addressing these root structural causes. First, I would ensure the tokenizer is explicitly trained on a comprehensive vocabulary that includes all necessary mathematical operators and decimal symbols to prevent physical blindness. Second, I would remove the strict 512-token memory cap and train the baseline model with a much larger sequence length, ensuring the model never learns to artificially truncate its thoughts. Finally, instead of jumping from 1-to-20 integers directly to extreme stress tests, I would implement a gradual scaling of the training data from 2-digit to 3-digit arithmetic, allowing the neural network to slowly form the proper weights and algorithmic carrying logic before facing massive OOD evaluations.

References

- [1] Nye, M., Andreassen, A. J., Gur-Ari, G., et al. (2021). Show Your Work: Scratchpads for Intermediate Computation with Language Models. *arXiv preprint arXiv:2112.00114*.
- [2] Dziri, N., Lu, X., Sclar, M., et al. (2023). Faith and Fate: Limits of Transformers on Compositionality. *Advances in Neural Information Processing Systems*, 36.

[3] Anil, C., Wu, Y., Andreassen, A., et al. (2022). Exploring Length Generalization in Large Language Models. *Advances in Neural Information Processing Systems*, 35.

Acknowledgment: During the preparation of this project, I utilized Google Gemini to assist in organizing thoughts, structuring the narrative flow of the report, and writing the matplotlib code to generate clear, high-quality visualizations. The AI was also used as a tool to help debug the evaluation scripts. I designed the experiments, actively conducted the analysis, and take full responsibility for the final code, charts, and conclusions presented in this report.